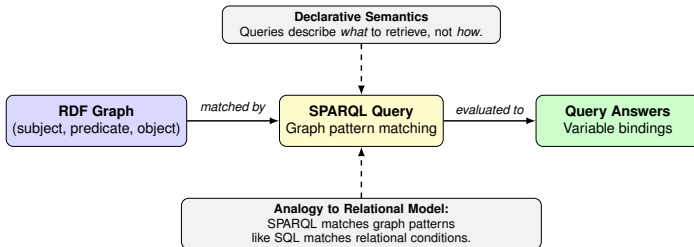


Knowledge Graphs & Query Processing (Lecture 3)



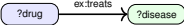
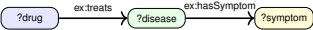
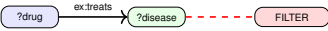
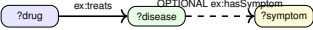
Knowledge Graph Query Processing



Agenda

- Graph Patterns
 - Triple Patterns
 - Basic Graph Patterns
 - Graph Patterns
 - Graph Patterns with FILTER
- SPARQL Query Language
 - Syntax
 - Semantics of SPARQL queries
 - Common errors in SPARQL queries

SPARQL Graph Patterns

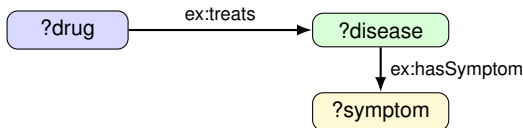
Pattern Type	Example Syntax	Visual Representation
Triple Pattern	<code>{ ?drug ex:treats ?disease }</code>	
Basic Graph Pattern (BGP)	<code>{ ?drug ex:treats ?disease . ?disease ex:hasSymptom ?symptom }</code>	
Graph Pattern with FILTER	<code>{ ?drug ex:treats ?disease . FILTER(?disease != ex:Cancer) }</code>	
Complex Graph Pattern	<code>{ ?drug ex:treats ?disease OPTIONAL(?disease ex:hasSymptom ?symptom) }</code>	

Graph Patterns [2]

Definition

Graph Patterns are combinations of *triple patterns* that define the structure of subgraphs to be matched in an RDF dataset.

- A **triple pattern** is the basic unit: (subject, predicate, object) where any can be a variable.
- A **graph pattern** combines multiple triple patterns that share variables.



A **graph pattern** = connected triple patterns sharing variables.

Triple Patterns

Given the infinite sets:

- $\mathcal{R}$ – set of **URIs**
- $\mathcal{B}$ – set of **Blank Nodes**
- $\mathcal{L}$ – set of **Literals**
- $\mathcal{Var}$ – set of **Variables**

Then a triple pattern is defined as:

$$\langle s, p, o \rangle \quad \text{where:} \quad \begin{cases} s \in (\mathcal{R} \cup \mathcal{B} \cup \mathcal{Var}) \\ p \in (\mathcal{R} \cup \mathcal{Var}) \\ o \in (\mathcal{R} \cup \mathcal{B} \cup \mathcal{L} \cup \mathcal{Var}) \end{cases}$$

Triple Patterns

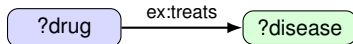
Given the infinite sets:

- $\mathcal{R}$ – set of **URIs**
- $\mathcal{B}$ – set of **Blank Nodes**
- $\mathcal{L}$ – set of **Literals**
- $\mathcal{Var}$ – set of **Variables**

Then a triple pattern is defined as:

$$\langle s, p, o \rangle \quad \text{where:} \quad \begin{cases} s \in (\mathcal{R} \cup \mathcal{B} \cup \mathcal{Var}) \\ p \in (\mathcal{R} \cup \mathcal{Var}) \\ o \in (\mathcal{R} \cup \mathcal{B} \cup \mathcal{L} \cup \mathcal{Var}) \end{cases}$$

Note: A variable is denoted by “?<label>”.



Example triple pattern: { ?drug ex:treats ?disease }

Examples of Triple Patterns (1/3)

Consider the RDF graph:



Q1 - The degrees of John Smith

- Triple pattern $t1 = \langle \text{ex:JohnSmith}, \text{ex:degree}, ?\text{major} \rangle$
- Evaluation in RDF graph G :

Examples of Triple Patterns (1/3)

Consider the RDF graph:



Q1 - The degrees of John Smith

- Triple pattern $t1 = \langle \text{ex:JohnSmith}, \text{ex:degree}, ?\text{major} \rangle$
- Evaluation in RDF graph G : **Note**: $?\text{major}$ is a variable.

Examples of Triple Patterns (1/3)

Consider the RDF graph:



Q1 - The degrees of John Smith

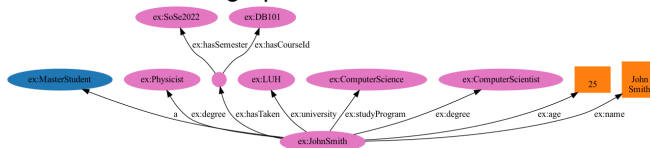
- Triple pattern $t1 = \langle \text{ex:JohnSmith}, \text{ex:degree}, ?\text{major} \rangle$
- Evaluation in RDF graph G : **Note**: $?major$ is a variable.

Result:

$\{ \{ (?major, \text{ex:ComputerScientist}), (?major, \text{ex:Physicist}) \} \}$

Examples of Triple Patterns (2/3)

Consider the RDF graph:



Q2 - The students of LUH

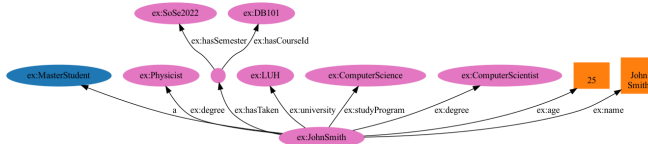
- Triple pattern $t2 = \langle ?student, ex:university, ex:LUH \rangle$
- Evaluation in RDF graph G :

Result:

$\{ \{ (?student, ex:JohnSmith) \} \}$

Examples of Triple Patterns (3/3)

Consider the RDF graph:



Q3 - The students and their relationship with LUH

- Triple pattern $t3 = \langle ?student, ?relation, ex:LUH \rangle$
- Evaluation in RDF graph G :

Result:

$\{ \{ (?student, ex:JohnSmith), (?relation, ex:university) \} \}$

Basic Graph Patterns (BGPs)

Given the infinite sets:

- $\mathcal{R}$ – set of **URLs**
- $\mathcal{B}$ – set of **Blank Nodes**
- $\mathcal{L}$ – set of **Literals**
- $\mathcal{Var}$ – set of **Variables**

Inductive Definition:

- **Base Case:** If $t = \langle s, p, o \rangle$ is a triple pattern, then t is a BGP.

Basic Graph Patterns (BGPs)

Given the infinite sets:

- $\mathcal{R}$ – set of **URIs**
- $\mathcal{B}$ – set of **Blank Nodes**
- $\mathcal{L}$ – set of **Literals**
- $\mathcal{Var}$ – set of **Variables**

Inductive Definition:

- **Base Case:** If $t = \langle s, p, o \rangle$ is a triple pattern, then t is a BGP.
- **Inductive Case:** If BGP_1 and BGP_2 are Basic Graph Patterns, then $BGP_1 \cdot BGP_2$ is a Basic Graph Pattern, where “ $\cdot$ ” denotes conjunction (logical JOIN).

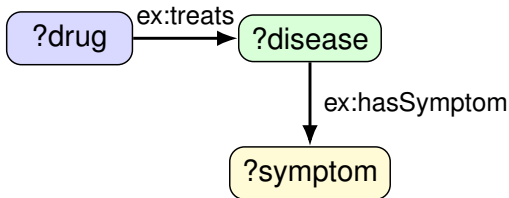
Basic Graph Patterns (BGP)

Given the infinite sets:

- $\mathcal{R}$ – set of **URLs**
- $\mathcal{B}$ – set of **Blank Nodes**
- $\mathcal{L}$ – set of **Literals**
- $\mathcal{V}$ – set of **Variables**

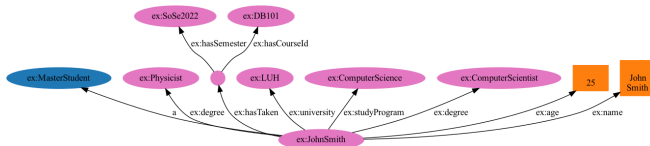
Inductive Definition:

- **Base Case:** If $t = \langle s, p, o \rangle$ is a triple pattern, then t is a BGP.
- **Inductive Case:** If BGP_1 and BGP_2 are Basic Graph Patterns, then $BGP_1 \cdot BGP_2$ is a Basic Graph Pattern, where “ $\cdot$ ” denotes conjunction (logical JOIN).



Examples of Basic Graph Pattern-Q4

Consider the RDF graph:



Q4 - The studyProgram of John Smith and his age

- Basic graph pattern:

```
bgp1 = <ex:JohnSmith    ex:studyProgram    ?major.
        ex:JohnSmith    ex:age                ?age>
```

- Evaluation in RDF graph:

```
Result: {(?major, ex:ComputerScience),
        (?age, "25"^^xsd:integer)}
```


Examples of Basic Graph Pattern-Q5

Q5 - The courses taken by the Master students of LUH and the semester when the course was taken.

- Basic graph pattern:

```

bgp2 = <?student      a                ex:MasterStudent.
        ?student      ex:university    ex:LUH.
        ?student      ex:hasTaken      ?courseTaken.
        ?courseTaken  ex:hasSemester   ?semester.
        ?courseTaken  ex:hasCourseID   ?course>
  
```

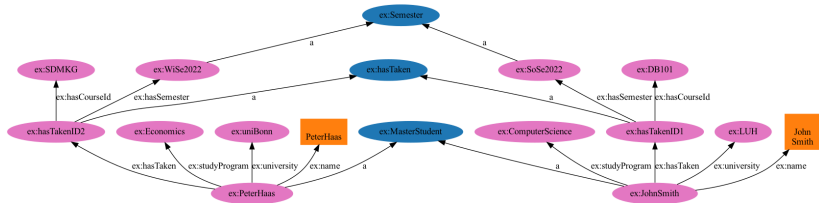
- Evaluation of bgp2 over RDF graph from slide 11:

```

Result: {(?student ,ex:JohnSmith),(?courseTaken,_,b11),
        (?semester,ex:SoSe2022),(?course,ex:DB101)}
  
```

Example RDF graph

Consider the RDF graph:



Examples of Basic Graph Pattern-Q6 (1/2)

Q6 - For each Master student, the university where the student is registered and the courses he/she is taken and the courses' semester.

- Basic graph pattern:

```
bgp3 = <?student      a                ex:MasterStudent.  
      ?student      ex:university ?university.  
      ?student      ex:hasTaken   ?courseTaken.  
      ?courseTaken  ex:hasSemester ?semester.  
      ?courseTaken  ex:hasCourseID ?course>
```

Examples of Basic Graph Pattern-Q6 (2/2)

Q6 - For each Master student, the university where the student is registered and the courses he/she is taken and the courses' semester.

- Evaluation of bgp3 over RDF graph from slide 13:

```
Result: {  
  {(?student,ex:JohnSmith),  
    (?university,ex:LUH),  
    (?courseTaken,ex:hasTakenID1),  
    (?semester,ex:SoSe2022),  
    (?course,ex:DB101)},  
  {(?student,ex:PeterHaas),  
    (?university,ex:UniBonn),  
    (?courseTaken,ex:hasTakenID2),  
    (?semester,ex:WiSe2022),  
    (?course,ex:SDMKG)}}}
```

Examples of Basic Graph Pattern-Q7 (1/2)

Q7 - For each Master student, the university where the student is registered and courses he/she is taken in this university and the study program

- Basic graph pattern:

```
bgp4 = <?student      a  ex:MasterStudent.  
      ?student      ex:university      ?university.  
      ?student      ex:hasTaken        ?courseTaken .  
      ?courseTaken  ex:hasSemester      ?semester.  
      ?courseTaken  ex:hasCourseID     ?course .  
      ?student      ex:studyProgram    ?program>
```

Examples of Basic Graph Pattern-Q7 (2/2)

Q7 - For each Master student, the university where the student is registered and courses he/she is taken in this university and the study program

- Evaluation of bgp4 over RDF graph from slide 13.:

```
Result: {  
  { (?student,ex:JohnSmith),(?university,ex:LUH),  
    (?courseTaken,ex:hasTakenID1),(?semester,ex:SoSe2022),  
    (?course,ex:DB101),(?program,ex:ComputerScience)},  
  {(?student,ex:PeterHaas),(?university,ex:UniBonn),  
    (?courseTaken,ex:hasTakenID2),(?semester,ex:WiSe2022),  
    (?course,ex:SDMKG),(?program,ex:Economics)}}}
```

Inductive Definition of a Graph Pattern (GP)

Given:

- The infinite sets:
 - R : set of URIs
 - B : set of Blank Nodes
 - L : set of Literals
- An infinite set Var : set of variables

Inductive Definition of a Graph Pattern (GP):

- **Base Case:** Let BGP be a basic graph pattern, then BGP is a Graph Pattern (GP).

Inductive Definition of a Graph Pattern (GP)

Given:

- The infinite sets:
 - R : set of URIs
 - B : set of Blank Nodes
 - L : set of Literals
- An infinite set Var : set of variables

Inductive Definition of a Graph Pattern (GP):

- **Base Case:** Let BGP be a basic graph pattern, then BGP is a Graph Pattern (GP).
- **Inductive Case:** If $GP1$ and $GP2$ are Graph Patterns, then:

$(GP1 * GP2)$ is a Graph Pattern GP

where $*$ $\in$ {UNION, OPT, .}:

- **UNION:** corresponds to the union of the graph patterns
- **OPT:** represents that $GP2$ is optional in $(GP1 \text{ OPT } GP2)$
- **“.”:** represents the AND (JOIN) of the graph patterns

Graph Patterns — *Rules*

Parenthesization: All expressions in SPARQL graph patterns are fully parenthesized.

Associativity of OPTIONAL (OPT)

$$(GP_1 \text{ OPT } (GP_2 \text{ OPT } GP_3)) \equiv ((GP_1 \text{ OPT } GP_2) \text{ OPT } GP_3)$$

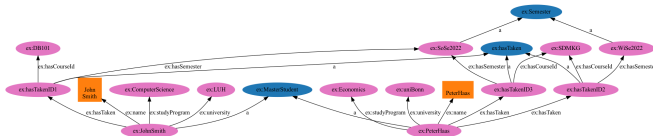
Precedence of AND over OPT

$$(GP_1 \text{ AND } GP_2 \text{ OPT } GP_3) \equiv ((GP_1 \text{ AND } GP_2) \text{ OPT } GP_3)$$

Parentheses clarify evaluation order in nested SPARQL graph patterns.

Examples of Graph Pattern-Q8 (1/2)

Consider the RDF graph:



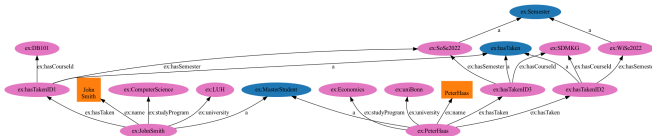
Q8 - “Master students from LUH or UniBonn”.

■ Basic graph pattern:

```
gp1 = <((?student a ex:MasterStudent.
           ?student ex:university ex:LUH)
  UNION
  (?student a ex:MasterStudent.
           ?student ex:university .ex:UniBonn))>
```

Examples of Graph Pattern-Q8 (2/2)

Consider the RDF graph:



Q8 - "Master students from LUH or UniBonn".

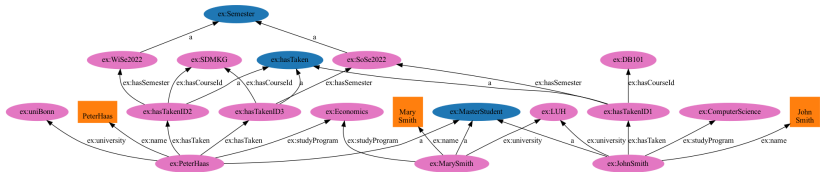
- Evaluation in RDF graph:

```
Result: {
```

$$\{(?student, ex:JohnSmith)\},$$
$$\{(?student, ex:PeterHaas)\}$$

Examples of Graph Pattern-Q9 (1/3)

Consider the RDF graph:



Examples of Graph Pattern-Q9 (2/3)

Q9 - “For each Master student, retrieve the university where the student is registered, and if he/she is taking any course, the information of these courses”.

■ Graph pattern:

```
gp1 = <((?student  a  ex:MasterStudent.  
    ?student  ex:university ?university.)  
    OPT  
    (?student      ex:hasTaken      ?courseTaken.  
    ?courseTaken  ex:hasSemester ?semester.  
    ?courseTaken  ex:hasCourseID ?course ))>
```

Examples of Graph Pattern-Q9 (3/3)

Q9 - “For each Master student, retrieve the university where the student is registered, and if he/she is taking any course, the information of these courses”.

- Evaluation in RDF graph in slide 22:

```
Result: {  
  {(?student,ex:JohnSmith),(?university,ex:LUH),  
    (?courseTaken,ex:hasTakenID1),(?semester,ex:SoSe2022),  
    (?course,ex:DB101)},  
  {(?student,ex:PeterHaas),(?university,ex:UniBonn),  
    (?courseTaken,ex:hasTakenID2),(?semester,ex:WiSe2022),  
    (?course,ex:SDMKG)},  
  {(?student,ex:MarySmith),(?university,ex:LUH)}  
}
```

Graph Patterns - with FILTER (1/2)

Given:

- The infinite sets:
 - R : set of URIs
 - B : set of Blank Nodes
 - L : set of Literals
- An infinite set Var : set of variables

Inductive Definition of a Graph Pattern (GP):

- **Base Case:** Let BGP be a basic graph pattern, then BGP is a Graph Pattern (GP).
- **Inductive Case:**
 - If $GP1$ and $GP2$ are Graph Patterns, then:

$(GP1 * GP2)$ is a Graph Pattern GP

where $*$ $\in$ {UNION, OPT, .}

Graph Patterns - with FILTER (2/2)

Inductive Definition of a Graph Pattern (GP) (Cont.):

- **Graph Patterns with FILTER:**

- If GP is a graph pattern and $Cond$ is a built-in condition, then:

$$GP \text{ FILTER } Cond$$

represents a graph pattern that selects only those results from GP that satisfy $Cond$.

- **Example:**

- Let GP be a pattern that retrieves all nodes.
- If $Cond$ is a condition that filters nodes by a specific property (e.g., label = "Person"), then:

$$GP \text{ FILTER (label = "Person")}$$

retrieves only nodes labeled as "Person".

Built-in Conditions (1/2)

Definitions:

- Given:
 - Infinitive sets:
 - R : set of URIs
 - B : set of Blank Nodes
 - L : set of literals
 - Var : set of variables
 - C : set of constants
 - $Lg = \{\neg, \wedge, \vee\}$: set of logical connectors
 - $CS = \{=, <, >, \geq, \leq\}$: set of comparison symbols

Base Case:

- If $?X$ and $?Y$ are variables (i.e., $?X \in Var$ and $?Y \in Var$), $c \in (R \cup B \cup L)$, and $sim \in CS$:
 - $(?X \text{ sim } c)$
 - $(?X \text{ sim } ?Y)$are built-in conditions.

Built-in Conditions (2/2)

Inductive Case:

- If R_1 and R_2 are built-in conditions, then:
 - $(\neg R_1)$
 - $(R_1 \vee R_2)$
 - $(R_1 \wedge R_2)$
- are built-in conditions.

Built-in Conditions - Rules

Variable Function:

- Let $\text{var}(\cdot)$ be a function that, given an expression E (i.e., graph pattern or built-in condition), returns the variables in E .

Rule:

- Given a graph pattern with filter ($GR \text{ FILTER } Cond$):

$$\text{var}(Cond) \subseteq \text{var}(GR)$$

Examples of Graph Pattern-Q10 (1/2)

Consider the RDF graph:



Examples of Graph Pattern-Q10 (1/2)

Consider the RDF graph:



Q10 - “For each Master student who is at least 25 years old, the university and the program where the student is registered”.

■ Graph pattern with FILTER:

```
gp1 = <((?student a ex:MasterStudent.
        ?student ex:university ?university .
        ?student ex:studyProgram ?program .
        ?student ex:age ?age .)
        FILTER (?age >='25'^^xsd:integer))>
```

Examples of Graph Pattern-Q10 (2/2)

Q10 - “For each Master student who is at least 25 years old, the university and the program where the student is registered”.

- Evaluation in RDF graph in slide 30:

```
Result: {  
  {(?student,ex:JohnSmith),(?university,ex:LUH),  
    (?program,ex:ComputerScience)},  
  {(?student,ex:MarySmith),(?university,ex:LUH),  
    (?program,ex:ComputerScience)},  
  {(?student,ex:PhilippLange),(?university,ex:UMD),  
    (?program,ex:ComputerScience)}  
}
```

SPARQL Overview

SPARQL – The Query Language for RDF

SPARQL (**SPARQL Protocol and RDF Query Language**) enables querying and manipulation of data stored as RDF triples, using graph-pattern matching semantics.

- What is SPARQL?
- SPARQL Syntax
- Example Queries
- SPARQL Semantics
- Complexity Results

SPARQL provides a formal bridge between RDF data and logical query evaluation.

What is SPARQL?



SPARQL 1.1 Query Language

W3C Recommendation 21 March 2013

This version:

<http://www.w3.org/TR/2013/REC-sparql11-query-20130321/>

Latest version:

<http://www.w3.org/TR/sparql11-query/>

Previous version:

<http://www.w3.org/TR/2012/PR-sparql11-query-20121108/>

Editors:

Steve Harris, Garlik, a part of Experian
Andy Seaborne, The Apache Software Foundation

Previous Editor:

Eric Prud'hommeaux, W3C

Please refer to the [errata](#) for this document, which may include some normative corrections.

See also [translations](#).

Copyright © 2013 W3C® (MIT, ERCIM, Keio, Beihang). All Rights Reserved. W3C [liability](#), [trademarks](#) and [document use](#) rules apply.

Abstract

RDF is a directed, labeled graph data format for representing information in the Web. This specification defines the syntax and semantics of the SPARQL query language for RDF. SPARQL can be used to express queries across diverse data sources, whether the data is stored natively as RDF or viewed as RDF via middleware. SPARQL contains capabilities for querying required and optional graph patterns along with their conjunctions and disjunctions. SPARQL also supports aggregation, subqueries, negation, creating values by expressions, extensible value testing, and constraining queries by source RDF graph. The results of SPARQL queries can be result sets or RDF graphs.

Status of This Document

May Be Superseded

This section describes the status of this document at the time of its publication. Other documents may supersede this document. A list of current W3C publications and the latest revision of this technical report can be found in the [W3C technical reports index](#) at <http://www.w3.org/TR/>.

Set of Documents

This document is one of eleven SPARQL 1.1 Recommendations produced by the [SPARQL Working Group](#):

1. [SPARQL 1.1 Overview](#)
2. [SPARQL 1.1 Query Language](#) (this document)
3. [SPARQL 1.1 Update](#)

<https://www.w3.org/TR/sparql11-query/>

Importance of SPARQL as a W3C Standard

SPARQL – A W3C-Recommended Query Language

- **SPARQL (SPARQL Protocol and RDF Query Language)** is the official W3C standard for querying and manipulating data represented in the **RDF**.
- It provides a **uniform and interoperable way** to retrieve and integrate linked data across the Web.

Importance of SPARQL as a W3C Standard

SPARQL – A W3C-Recommended Query Language

- **SPARQL (SPARQL Protocol and RDF Query Language)** is the official W3C standard for querying and manipulating data represented in the **RDF**.
- It provides a **uniform and interoperable way** to retrieve and integrate linked data across the Web.

Why SPARQL's Standardization Matters

- ★ **Interoperability:** Widely supported across RDF stores and tools, ensuring cross-platform compatibility.
- ★ **Data Integration:** Enables federated queries across heterogeneous data sources.
- ★ **Innovation & Trust:** As a W3C standard, it provides a stable foundation for innovative semantic-web applications.

SPARQL Basics

PREFIX <prefix-declarations>
SELECT <Var-List>
FROM <Graph-IRIs> (*optional*)
WHERE { <Graph-Pattern> }
FILTER (<Condition>) (*optional*)
ORDER BY / LIMIT <Ordering / N> (*optional*)

SPARQL Basics

PREFIX <prefix-declarations>
SELECT <Var-List>
FROM <Graph-IRIs> (optional)
WHERE { <Graph-Pattern> }
FILTER (<Condition>) (optional)
ORDER BY / LIMIT <Ordering / N> (optional)

- <Var-List> are the variables to return (e.g., ?drug ?disease).
- <Graph-IRIs> optionally selects named graphs/datasets.
- <Graph-Pattern> is a set of triple patterns and operators (e.g., ., OPTIONAL, FILTER).
- <Condition> constrains bindings (e.g., ?disease != ex:Cancer).
- <Ordering / N> controls result order/size (e.g., ORDER BY ?drug, LIMIT 5).

SPARQL- Syntax SELECT queries (1/3)

```
<Select-From-Where-Query> :=  
[<Namespaces>]  
SELECT <Projected_variables>  
[FROM <RDF_dataset>]*  
WHERE <Graph_pattern_filter>
```

```
<Namespaces>:= PREFIX <name>: <URL>  
<Namespaces>:= <Namespaces> <Namespaces>
```

```
<Projected_variables>:= <DISTINCT> <List_vars>  
<DISTINCT>:= Empty| DISTINCT  
<List_vars>:= <Vars> | *  
<Vars> := <Var>  
<Vars> := <Vars> <Vars>
```

SPARQL- Syntax SELECT queries (1/3)

<Select-From-Where-Query> :=

[<Namespaces>]

SELECT <Projected_variables>

[FROM <RDF_dataset>]*

WHERE <Graph_pattern_filter>

<Namespaces>:= PREFIX <name>: <URL>

<Namespaces>:= <Namespaces> <Namespaces>

<Projected_variables>:= <DISTINCT> <List_vars>

<DISTINCT>:= Empty| DISTINCT

<List_vars>:= <Vars> | *

<Vars> := <Var>

<Vars> := <Vars> <Vars>

Note: "*" in <List_vars> production rule represents that all variables in the WHERE clause of the query are projected.

SPARQL- Syntax SELECT queries (2/3)

`<Graph_pattern_filter>:= {<Graph_pattern>}. <Filter>`

`<Filter>:= Empty`

`<Filter>:= FILTER(<Built-in_Conditions>) <Filter>`

`<Built-in_Conditions>:= (<Built-in_Conditions>)`

`<Built-in_Conditions>:= <SimpleComparison>`

`<Built-in_Conditions>:=`

`<Built-in_Conditions> <BinaryLogOP> <Built-in_Conditions>`

`<Built-in_Conditions>:= <UnaryLogOP> <Built-in_Conditions>`

`<SimpleComparison>:= <VAR> <Comp> <Value>`

`<Value>:= <VAR> | <Constant> | <URI> | <BlankNode>`

`<Comp> := = | < | > | >= | <=`

`<UnaryLogOP>:= NOT`

`<BinaryLogOP>:= AND | OR`

SPARQL- Syntax SELECT queries (3/3)

`<Graph_pattern>:= {<Graph_pattern>}`

`<Graph_pattern>:=`

`<Graph_pattern> OPTIONAL <Graph_pattern>`

`<Graph_pattern>:=`

`<Graph_pattern> UNION <Graph_pattern>`

`<Graph_pattern>:=`

`<Graph_pattern> . <Graph_pattern>`

`<Graph_pattern>:= <Triple_pattern>`

`<Triple_pattern>:= <Subject> <Property> <Object>`

`<Subject>:= <VAR> | <URI> | <BlankNode>`

`<Property>:= <VAR> | <URI>`

`<Object>:= <VAR> | <URI> | <BlankNode> | <Literal>`

Example- RDF Graph

```
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

ex:JohnSmith rdf:type          ex:MasterStudent;
              ex:studyProgram  ex:ComputerScience ;
              ex:name          "John Smith" ;
              ex:degree        ex:Physicist , ex:ComputerScientist ;
              ex:university    ex:LUH;
              ex:hasTaken      ex:hasTakenID1.

ex:MarySmith  rdf:type          ex:MasterStudent;
              ex:studyProgram  ex:Economics ;
              ex:name          "Mary Smith" ;
              ex:university    ex:LUH.

ex:PeterHaas  rdf:type          ex:MasterStudent;
              ex:studyProgram  ex:Economics ;
              ex:name          "PeterHaas" ;
              ex:university    ex:UniBonn;
              ex:hasTaken      ex:hasTakenID3, ex:hasTakenID2 .

ex:hasTakenID1 rdf:type          ex:hasTaken ;
               ex:hasCourseId  ex:DB101;
               ex:hasSemester  ex:SoSe2024 .

ex:hasTakenID2 rdf:type          ex:hasTaken ;
               ex:hasCourseId  ex:SDMKG;
               ex:hasSemester  ex:WiSe2024 .

ex:hasTakenID3 rdf:type          ex:hasTaken ;
               ex:hasCourseId  ex:SDMKG;
               ex:hasSemester  ex:SoSe2024 .
```

Example- Triple pattern queries (1/3)

Consider the RDF graph in slide 39:

QS1 - The degrees of John Smith

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?degree
WHERE {ex:JohnSmith ex:degree ?degree}
```

Example- Triple pattern queries (1/3)

Consider the RDF graph in slide 39:

QS1 - The degrees of John Smith

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?degree
WHERE {ex:JohnSmith ex:degree ?degree}
```

- PREFIX: Defines shorthand ex: for http://example.org/
- SELECT DISTINCT: Returns unique values only
- WHERE: Specifies the triple pattern to match:
 - Subject: ex:JohnSmith
 - Predicate: ex:degree
 - Object: Variable ?degree (what we are looking for)

Example- Triple pattern queries (2/3)

Consider the RDF graph in slide 39:

QS2 - The students of LUH

```
PREFIX ex: <http://example.org/>  
SELECT DISTINCT ?student  
WHERE {?student ex:university ex:LUH}
```

Example- Triple pattern queries (2/3)

Consider the RDF graph in slide 39:

QS2 - The students of LUH

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?student
WHERE {?student ex:university ex:LUH}
```

- PREFIX: Defines shorthand ex: for http://example.org/
- SELECT DISTINCT: Returns unique values only
- WHERE: Specifies the triple pattern to match:
 - Subject: Variable ?student (what we are looking for)
 - Predicate: ex:university
 - Object: ex:LUH (Leibniz University Hannover)

Example- Triple pattern queries (3/3)

Consider the RDF graph in slide 39:

QS3 - What types of relationships exist between students and LUH?

```
PREFIX ex: <http://example.org/>  
SELECT DISTINCT ?student ?relationship  
WHERE {?student ?relationship ex:LUH}
```

Example- Triple pattern queries (3/3)

Consider the RDF graph in slide 39:

QS3 - What types of relationships exist between students and LUH?

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?student ?relationship
WHERE {?student ?relationship ex:LUH}
```

- PREFIX: Defines shorthand ex: for http://example.org/
- SELECT DISTINCT: Returns unique students and their relationships
- WHERE: Specifies the triple pattern to match:
 - Subject: Variable ?student (what we are looking for)
 - Predicate: Variable ?relationship (type of connection to LUH)
 - Object: ex:LUH (Leibniz University Hannover)

Example- Basic graph pattern queries (1/3)

Consider the RDF graph in slide 39:

QS4 - The study program of John Smith and his university

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?program ?university
WHERE {ex:JohnSmith ex:studyProgram ?program.
       ex:JohnSmith ex:university ?university.}
```


Example- Basic graph pattern queries (1/3)

Consider the RDF graph in slide 39:

QS4 - The study program of John Smith and his university

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?program ?university
WHERE {ex:JohnSmith ex:studyProgram ?program.
       ex:JohnSmith ex:university ?university.}
```

- WHERE: Contains a basic graph pattern with two triple patterns:
 - First triple pattern:
 - Subject: `ex:JohnSmith`
 - Predicate: `ex:studyProgram`
 - Object: Variable `?program` (program)
 - Second triple pattern:
 - Subject: `ex:JohnSmith`
 - Predicate: `ex:university`
 - Object: Variable `?university` (university)
 - The period (.) between patterns indicates AND logic

Example- Basic Graph Pattern Queries (2/3)

Consider the RDF graph in slide 39:

QS5 - The courses taken by the Master students of LUH and the semester when the course was taken

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {?student      a                ex:MasterStudent.
      ?student      ex:university    ex:LUH.
      ?student      ex:hasTaken      ?courseTaken.
      ?courseTaken  ex:hasSemester   ?semester .
      ?courseTaken  ex:hasCourseId   ?course}
```

Example- Basic Graph Pattern Queries (2/3)

Consider the RDF graph in slide 39:

QS5 - The courses taken by the Master students of LUH and the semester when the course was taken

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {?student      a                ex:MasterStudent.
       ?student      ex:university    ex:LUH.
       ?student      ex:hasTaken      ?courseTaken.
       ?courseTaken  ex:hasSemester   ?semester .
       ?courseTaken  ex:hasCourseId   ?course}
```

- *: Represents all the variables in the WHERE condition are projected.
- WHERE: Contains a basic graph pattern comprising four triple patterns.
- The period (.) between patterns indicates AND logic

Example - Basic Graph Pattern Queries (3/3)

Consider the RDF graph in slide 39:

QS6 - Retrieve the universities where master's students are enrolled, the courses they have taken, and the semesters for these courses.

```
PREFIX ex: <http://example.org/>
```

```
SELECT DISTINCT *
```

```
WHERE {
```

```
    ?student      a                ex:MasterStudent.
```

```
    ?student      ex:university    ?university.
```

```
    ?student      ex:hasTaken      ?courseTaken.
```

```
    ?courseTaken  ex:hasSemester   ?semester.
```

```
    ?courseTaken  ex:hasCourseId   ?course
```

```
}
```

Example - Basic Graph Pattern Queries (3/3)

Consider the RDF graph in slide 39:

QS6 - Retrieve the universities where master's students are enrolled, the courses they have taken, and the semesters for these courses.

```
PREFIX ex: <http://example.org/>
```

```
SELECT DISTINCT *
```

```
WHERE {
```

```
    ?student      a                ex:MasterStudent.
```

```
    ?student      ex:university    ?university.
```

```
    ?student      ex:hasTaken      ?courseTaken.
```

```
    ?courseTaken  ex:hasSemester   ?semester.
```

```
    ?courseTaken  ex:hasCourseId   ?course
```

```
}
```

- WHERE: Contains a basic graph pattern comprising four triple patterns.
- The period (.) between patterns indicates AND logic, meaning all conditions must be met simultaneously.

Example- Graph pattern queries (1/3)

Consider the RDF graph in slide 39:

QS7 - Master students from LUH or UniBonn

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{?student a ex:MasterStudent .
        ?student ex:university ex:LUH.}
        UNION
        {?student a ex:MasterStudent .
        ?student ex:university ex:UniBonn.}}
```

Example- Graph pattern queries (1/3)

Consider the RDF graph in slide 39:

QS7 - Master students from LUH or UniBonn

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{?student a ex:MasterStudent .
        ?student ex:university ex:LUH.}
        UNION
        {?student a ex:MasterStudent .
        ?student ex:university ex:UniBonn.}}
```

- WHERE: Contains two graph patterns connected by UNION.
- UNION operator: Combines results from both patterns (OR logic)
- The period (.) between patterns indicates AND logic

Example- Graph Pattern Queries (2/3)

Consider the RDF graph in slide 39:

QS8 - Retrieve the university where students are registered, and if they are taking any course, the information of these courses

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{?student a ex:MasterStudent .
        ?student ex:university ?university.}
        OPTIONAL
        {?student ex:hasTaken ?courseTaken .
        ?courseTaken ex:hasSemester ?semester .
        ?courseTaken ex:hasCourseId ?course .}}}
```


Example- Graph Pattern Queries (2/3)

Consider the RDF graph in slide 39:

QS8 - Retrieve the university where students are registered, and if they are taking any course, the information of these courses

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{?student a ex:MasterStudent .
       ?student ex:university ?university.}
       OPTIONAL
       {?student ex:hasTaken ?courseTaken .
        ?courseTaken ex:hasSemester ?semester .
        ?courseTaken ex:hasCourseId ?course .}}
```

- OPTIONAL keyword: Includes course information if it exists, but still returns results even if no courses are found
- Results will include:
 - All master students and their universities (required)
 - Course information only for students who are enrolled in courses

Example- Graph pattern queries (3/3)

Consider the RDF graph in slide 39:

QS9 - For students of LUH or UniBonn, retrieve their study program, and if they are taking any course, retrieve the information of these courses

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{{?student a ex:MasterStudent .
        ?student ex:university ex:LUH}
      UNION
        {?student a ex:MasterStudent .
          ?student ex:university ex:UniBonn}}}
OPTIONAL
  {?student      ex:hasTaken      ?courseTaken .
    ?courseTaken ex:hasSemester ?semester .
    ?courseTaken ex:hasCourseId ?course .}}
```

More Syntax- GROUP BY

GROUP BY allows for grouping results based on specified variables, enabling aggregation of data in SPARQL queries.

```
<Query_Grouping>:=  
<Select-From-Where-Query>  
GROUP BY <G-Expression>
```

```
<Select-From-Where-Query> :=  
<Namespaces>  
SELECT <Projected_Expressions>  
[FROM <RDF_dataset>]*  
WHERE <Graph_pattern_filter>
```

More Syntax- GROUP BY

GROUP BY allows for grouping results based on specified variables, enabling aggregation of data in SPARQL queries.

```
<Query_Grouping>:=  
<Select-From-Where-Query>  
GROUP BY <G-Expression>
```

```
<Select-From-Where-Query> :=  
<Namespaces>  
SELECT <Projected_Expressions>  
[FROM <RDF_dataset>]*  
WHERE <Graph_pattern_filter>  
  
<G-Expression>:= <Vars>
```

More Syntax- GROUP BY

GROUP BY allows for grouping results based on specified variables, enabling aggregation of data in SPARQL queries.

```
<Query_Grouping>:=  
<Select-From-Where-Query>  
GROUP BY <G-Expression>
```

```
<Select-From-Where-Query> :=  
<Namespaces>  
SELECT <Projected_Expressions>  
[FROM <RDF_dataset>]*  
WHERE <Graph_pattern_filter>
```

```
<G-Expression>:= <Vars>
```

GROUP BY Requirement: In SPARQL, as in SQL, any variables used in the **GROUP BY** clause must also appear in the **SELECT** clause. This ensures that the query is logically consistent and that all grouped data can be represented in the query's output.

More Syntax- Aggregated Expressions

Aggregated Expressions in SPARQL allow for calculations on grouped data, including operations like counting, summing, and averaging.

```
<Projected_Expressions>:= <Expression>
<Projected_Expressions>:=
<Projected_Expressions> <Projected_Expressions>
<Expression>:= <Projected_Variables>|<Aggregates>
<Aggregates>:= <Aggregate>
<Aggregates>:=<Aggregates> <Aggregates>
<Aggregate>:= COUNT(<Var>)| SUM(<Var>) | AVG (<Var>) |
MIN (<Var>) | MAX (<Var>)
```

Example- GROUP BY and Aggregated Expressions

Consider the RDF graph in slide 39:

QS10 - Retrieve the number of master students per university

```
PREFIX ex: <http://example.org/>
SELECT ?university (COUNT(DISTINCT ?student) as ?NumStudents)
WHERE {?student a                ex:MasterStudent .
       ?student ex:university ?university}
GROUP BY ?university
```

Example- GROUP BY and Aggregated Expressions

Consider the RDF graph in slide 39:

QS10 - Retrieve the number of master students per university

```
PREFIX ex: <http://example.org/>
SELECT ?university (COUNT(DISTINCT ?student) as ?NumStudents)
WHERE {?student a                ex:MasterStudent .
       ?student ex:university ?university}
GROUP BY ?university
```

Explanation:

- ?university appears in both the GROUP BY and SELECT clauses.
- This query groups master students by university and retrieves each university with the number of unique students.
- If a variable is in the GROUP BY clause but missing from SELECT, SPARQL will raise an error, as the output would be incomplete.

SPARQL Syntax - ORDER BY

ORDER BY is used in SPARQL to sort query results based on one or more variables.

Syntax Definition:

```
<Query_Sorting> :=  
  <Select-From-Where-Group-Query>  
  ORDER BY <Order_Expression>
```

```
<Select-From-Where-Group-Query> :=  
  <Namespaces>  
  SELECT <Projected_Expressions>  
  [FROM <RDF_dataset>]*  
  WHERE <Graph_pattern_filter>  
  [GROUP BY <G-Expression>]
```

```
<Order_Expression> := ASC(?Var) | DESC(?Var) | ?Var
```

SPARQL Syntax - ORDER BY

ORDER BY is used in SPARQL to sort query results based on one or more variables.

Syntax Definition:

```
<Query_Sorting> :=  
  <Select-From-Where-Group-Query>  
  ORDER BY <Order_Expression>
```

```
<Select-From-Where-Group-Query> :=  
  <Namespaces>  
  SELECT <Projected_Expressions>  
  [FROM <RDF_dataset>]*  
  WHERE <Graph_pattern_filter>  
  [GROUP BY <G-Expression>]
```

```
<Order_Expression> := ASC(?Var) | DESC(?Var) | ?Var
```

Note:

- The **ORDER BY** clause can be used to sort results in ascending ('ASC') or descending ('DESC') order.
- Multiple variables can be specified, and they will be sorted in the order they appear.

Example — **GROUP BY** and **ORDER BY**

QS11 – Retrieve the number of master students per university, sorted by the number of students in descending order.

SPARQL Query

```
PREFIX ex: <http://example.org/>
SELECT ?university (COUNT(?student) AS ?NumStudents)
WHERE {
    ?student a ex:MasterStudent .
    ?student ex:university ?university .}
GROUP BY ?university
ORDER BY DESC(?NumStudents)
```

Example — **GROUP BY** and **ORDER BY**

QS11 – Retrieve the number of master students per university, sorted by the number of students in descending order.

SPARQL Query

```
PREFIX ex: <http://example.org/>
SELECT ?university (COUNT(?student) AS ?NumStudents)
WHERE {
    ?student a ex:MasterStudent .
    ?student ex:university ?university .}
GROUP BY ?university
ORDER BY DESC(?NumStudents)
```

Explanation:

- Groups all ?student resources by their corresponding ?university.
- COUNT(): counts how many master students belong to each university.
- ORDER BY DESC(?NumStudents) sorts universities by student count in descending order.

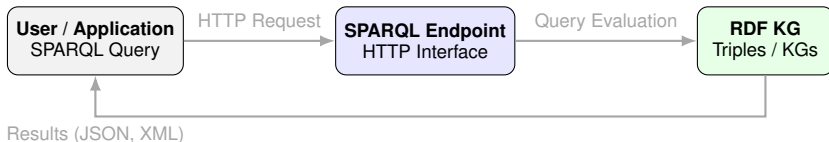
SPARQL Endpoints (1/2)

Definition

A **SPARQL Endpoint** is a web-accessible service that allows users to query an RDF dataset using the **SPARQL** query language.

Main Idea:

- Acts as a **gateway** between users and RDF knowledge graphs.
- Accepts SPARQL queries (e.g., SELECT, ASK, CONSTRUCT, DESCRIBE).
- Returns results in standard formats such as JSON, XML, or CSV.



SPARQL Endpoints make RDF data accessible through web-based query interfaces.

SPARQL Endpoints (2/2)

Key Features

- **Remote Querying:** Send queries via HTTP and retrieve results from anywhere.
- **Standardized API:** Uses W3C protocols, ensuring interoperability across systems.
- **Data Discovery:** Enables exploration of interconnected Linked Open Data.
- **Access Models:** Endpoints may be public (e.g., Wikidata) or private (institutional).

Examples of Public Endpoints

- <https://query.wikidata.org/> – Wikidata
- <https://bio2rdf.org/sparql> – Bio2RDF
- <https://dbpedia.org/sparql> – DBpedia

SPARQL Endpoints connect distributed RDF data to researchers, tools, and applications.

Exercise

Participate **in the survey at Stud.IP.**

Query over Wikidata

Consider the following SPARQL query to be executed over the SPARQL endpoint: <https://query.wikidata.org/>

```
SELECT (COUNT(*) AS ?count)
WHERE {
    ?item wdt:P31 wd:Q5 .
}
```

Explanation of Terms:

- wdt:P31 represents the predicate "instance of."
- wd:Q5 corresponds to the class "human" in Wikidata.

Query Description:

QR1 - Retrieve the total count of items in Wikidata that are instances of the class "human."

Query over DBpedia

Consider the following SPARQL query to be executed over the SPARQL endpoint: <https://dbpedia.org/sparql>

SPARQL Query:

```
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbp: <http://dbpedia.org/property/>
SELECT ?excretion ?metabolism (COUNT(DISTINCT ?drug) AS ?Num)
WHERE {
    ?drug a dbo:Drug .
    ?drug dbp:excretion ?excretion .
    ?drug dbp:metabolism ?metabolism .}
GROUP BY ?excretion ?metabolism
ORDER BY DESC(?Num)
```

Query Description:

QR2 - Count the number of drugs grouped by metabolism and excretion, displaying the results in descending order by the count.

Query over the Bio2RDF KG

Consider the following SPARQL query to be execute over the SPARQL endpoint: <https://bio2rdf.org/sparql>

```
PREFIX bio2rdf: <http://bio2rdf.org/pharmgkb_vocabulary:>
PREFIX purl: <http://purl.org/dc/terms/>
PREFIX voc: <http://bio2rdf.org/drugbank_vocabulary:>
SELECT ?drug ?title ?indication
WHERE {?drug      a                bio2rdf:Drug .
       ?drug      purl:title       ?title .
       ?drug      bio2rdf:x-drugbank ?drugbank .
       ?drugbank  voc:indication   ?indication . }
```

Query Description:

QR3 - For each drug in the BioRDF knowledge graph, retrieve its title and indication.

Semantics and Complexity of SPARQL [1, 3]

Semantics and Complexity of SPARQL

JORGE PÉREZ and MARCELO ARENAS

Pontificia Universidad Católica de Chile
and

CLAUDIO GUTIERREZ
Universidad de Chile

SPARQL is the standard language for querying RDF data. In this article, we address systematically the formal study of the database aspects of SPARQL, concentrating in its graph pattern matching facility. We provide a compositional semantics for the core part of SPARQL, and study the complexity of the evaluation of several fragments of the language. Among other complexity results, we show that the evaluation of general SPARQL patterns is PSPACE-complete. We identify a large class of SPARQL patterns, defined by imposing a simple and natural syntactic restriction, where the query evaluation problem can be solved more efficiently. This restriction gives rise to the class of well-designed patterns. We show that the evaluation problem is coNP-complete for well-designed patterns. Moreover, we provide several rewriting rules for well-designed patterns whose application may have a considerable impact in the cost of evaluating SPARQL queries.

Categories and Subject Descriptors: H.2.3 [Database Management]: Languages—Query languages

General Terms: Algorithms, Languages, Performance, Theory

Additional Key Words and Phrases: Complexity, query language, RDF, semantic Web, SPARQL

ACM Reference Format:

Pérez, J., Arenas, M., and Gutierrez, C. 2009. Semantics and complexity of SPARQL. ACM Trans. Database Syst. 34, 3, Article 16 (August 2009), 45 pages.
DOI = 10.1145/1567274.1567278 <http://doi.acm.org/10.1145/1567274.1567278>

1. INTRODUCTION

The Resource Description Framework (RDF) [Manola and Miller 2004] is a data model for representing information about World Wide Web resources. Jointly

This article is an extended and revised version of Pérez et al. [2006a].

M. Arenas was supported by Fondecyt grants 1070732 and 1090565; C. Gutierrez was supported by Fondecyt grant 1070348; J. Pérez was supported by Conicyt Ph.D. Scholarship.

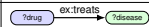
Authors' addresses: J. Pérez, M. Arenas, Department of Computer Science, Pontificia Universidad Católica de Chile, Chile; email: {jperez,ma Arenas}@ing.puc.cl; C. Gutierrez, Department of Computer Science, Universidad de Chile, Chile; email: cgutierrez@dcc.uchile.cl.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 889-0481, or permissions@acm.org.
© 2009 ACM 0362-5915/09/08-ART16 \$10.00
DOI 10.1145/1567274.1567278 <http://doi.acm.org/10.1145/1567274.1567278>

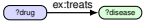
ACM Transactions on Database Systems, Vol. 34, No. 3, Article 16, Publication date: August 2009.

<https://dl.acm.org/doi/pdf/10.1145/1567274.1567278>

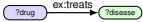
SPARQL SELECT Queries – Concepts and Syntax

SPARQL Concept	Example Syntax WHERE clause	
Triple Pattern	?drug ex:treats ?disease	

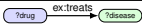
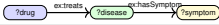
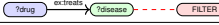
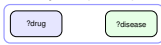
SPARQL SELECT Queries – Concepts and Syntax

SPARQL Concept	Example Syntax WHERE clause	
Triple Pattern	<code>?drug ex:treats ?disease</code>	
Basic Graph Pattern (BGP)	<code>{ ?drug ex:treats ?disease . ?disease ex:hasSymptom ?symptom }</code>	

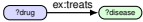
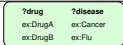
SPARQL SELECT Queries – Concepts and Syntax

SPARQL Concept	Example Syntax WHERE clause	
Triple Pattern	<code>?drug ex:treats ?disease</code>	
Basic Graph Pattern (BGP)	<code>{ ?drug ex:treats ?disease . ?disease ex:hasSymptom ?symptom }</code>	
FILTER Clause	<code>FILTER(?disease != ex:Cancer)</code>	

SPARQL SELECT Queries – Concepts and Syntax

SPARQL Concept	Example Syntax WHERE clause	
Triple Pattern	<code>?drug ex:treats ?disease</code>	
Basic Graph Pattern (BGP)	<code>{ ?drug ex:treats ?disease . ?disease ex:hasSymptom ?symptom }</code>	
FILTER Clause	<code>FILTER(?disease != ex:Cancer)</code>	
Projection (SELECT)	<code>SELECT ?drug ?disease</code>	Projection (SELECT)  Only variables listed in the SELECT clause appear in the results.

SPARQL SELECT Queries – Concepts and Syntax

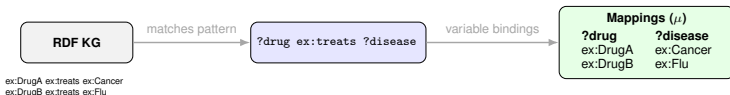
SPARQL Concept	Example Syntax WHERE clause	
Triple Pattern	<code>?drug ex:treats ?disease</code>	
Basic Graph Pattern (BGP)	<code>{ ?drug ex:treats ?disease . ?disease ex:hasSymptom ?symptom }</code>	
FILTER Clause	<code>FILTER(?disease != ex:Cancer)</code>	
Projection (SELECT)	<code>SELECT ?drug ?disease</code>	Projection (SELECT)  Only variables listed in the SELECT clause appear in the results.
Resulting Mappings	<code>(?drug, ?disease)</code>	

Mappings of Variables

Intuition

A **mapping** connects the *variables* in a SPARQL query with the *values* found in the RDF KG that make the query true.

Example:



A mapping is one assignment of RDF values to query variables that satisfies the pattern.

Semantics of Triple Pattern SPARQL Queries

Variable and Triple Pattern Mappings:

- Let **Var** be a set of variables, **U** a set of URIs, **L** a set of literals, and **B** a set of blank nodes.
- A **variable mapping** μ is a partial function:

$$\mu : \text{Var} \rightarrow U \cup L \cup B$$

Semantics of Triple Pattern SPARQL Queries

Variable and Triple Pattern Mappings:

- Let **Var** be a set of variables, **U** a set of URIs, **L** a set of literals, and **B** a set of blank nodes.
- A **variable mapping** μ is a partial function:

$$\mu : \text{Var} \rightarrow U \cup L \cup B$$

- **Triple Pattern Mappings:** For a triple pattern tp , the mapping $\mu(tp)$ is defined as:

$$\mu(tp) = \{(v, \mu(v)) \mid v \in \text{vars}(tp) \text{ and } \mu(v) \text{ is defined}\}$$

- Where $\text{vars}(tp) = \{u \mid u \text{ is a variable in } tp\}$

Example - Triple Pattern SPARQL Query

Consider the RDF graph:



Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?program
WHERE { ex:JohnSmith ex:studyProgram ?program }
```

Example - Triple Pattern SPARQL Query

Consider the RDF graph:



Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?program
WHERE { ex:JohnSmith ex:studyProgram ?program }
```

Explanation:

- This query retrieves unique values for `?program` linked to `ex:JohnSmith` through the predicate `ex:studyProgram`.
- `?program` is a variable bound to objects associated with `ex:JohnSmith` in the RDF graph.

Summary of SPARQL Semantics

Graph Pattern Type	Semantics / Interpretation
Triple Pattern	Evaluated over RDF graph G . Returns all mappings μ such that $\mu(t) \in G$ $[[t]]_G = \{\mu \mid \mu(t) \in G\}$
Basic Graph Pattern (BGP)	Evaluated as a join of the mappings of its triple patterns: $[[t_1 \text{ AND } t_2]]_G = [[t_1]]_G \bowtie [[t_2]]_G$
FILTER	Keeps only mappings satisfying a Boolean condition R : $[[P \text{ FILTER } R]]_G = \{\mu \in [[P]]_G \mid \mu \models R\}$
UNION	Combines all mappings from both patterns: $[[P_1 \text{ UNION } P_2]]_G = [[P_1]]_G \cup [[P_2]]_G$
MINUS	Removes mappings from P_1 that are compatible with any in P_2 : $[[P_1 \text{ MINUS } P_2]]_G = \{\mu \in [[P_1]]_G \mid \forall \mu' \in [[P_2]]_G, \mu \text{ and } \mu' \text{ incompatible}\}$
OPTIONAL	Keeps all mappings from P_1 ; adds matches from P_2 when possible: $[[P_1 \text{ OPTIONAL } P_2]]_G = ([[P_1]]_G \bowtie [[P_2]]_G) \cup ([[P_1]]_G \setminus [[P_2]]_G)$

SPARQL semantics define how graph patterns are matched and combined to produce variable bindings.

Semantics of SPARQL Triple Patterns

Evaluating Triple Patterns:

- Given a triple pattern t where $t = (s, p, o)$ and an RDF graph G .
- The semantics of t with respect to G , denoted $[[t]]_G$, is defined as:

$$[[t]]_G = \{\mu(t) \mid \text{dom}(\mu) = \text{vars}(t) \text{ and } \mu(t) \in G\}$$

- Here, $\text{dom}(\mu)$ is the subset of **Var** where μ is defined.

Semantics of SPARQL Triple Patterns

Evaluating Triple Patterns:

- Given a triple pattern t where $t = (s, p, o)$ and an RDF graph G .
- The semantics of t with respect to G , denoted $[[t]]_G$, is defined as:

$$[[t]]_G = \{\mu(t) \mid \text{dom}(\mu) = \text{vars}(t) \text{ and } \mu(t) \in G\}$$

- Here, $\text{dom}(\mu)$ is the subset of **Var** where μ is defined.

Interpretation:

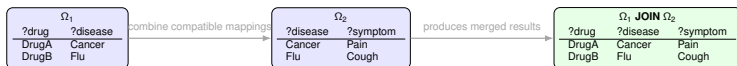
- This means $[[t]]_G$ returns all mappings μ that satisfy the triple pattern t within the graph G .

JOIN and Compatibility of Mappings

Intuition

The evaluation of **Basic Graph Pattern (BGP)** combines mappings from different triple patterns, using a **JOIN** for merging compatible mappings.

Example:



Compatibility: Two mappings are compatible if they assign the **same value** to shared variables (here, ?disease).

JOIN merges mappings from different triple patterns based on shared variable values.

Semantics of Basic Graph Patterns

Definition: JOIN Operation

- Let Ω_1 and Ω_2 be sets of mappings.
- The JOIN of Ω_1 and Ω_2 is defined as:

$$\Omega_1 \text{ JOIN } \Omega_2 =$$

$$\{\mu_1 \cup \mu_2 \mid \mu_1 \in \Omega_1 \text{ and } \mu_2 \in \Omega_2 \text{ and } \mu_1 \text{ and } \mu_2 \text{ are compatible}\}$$

Semantics of Basic Graph Patterns

Definition: JOIN Operation

- Let Ω_1 and Ω_2 be sets of mappings.
- The JOIN of Ω_1 and Ω_2 is defined as:
$$\Omega_1 \text{ JOIN } \Omega_2 = \{ \mu_1 \cup \mu_2 \mid \mu_1 \in \Omega_1 \text{ and } \mu_2 \in \Omega_2 \text{ and } \mu_1 \text{ and } \mu_2 \text{ are compatible} \}$$

Compatibility of Mappings

- Let μ_1 and μ_2 be mappings.
- μ_1 and μ_2 are **compatible** iff:

$$\forall ?X \in \text{dom}(\mu_1) \cap \text{dom}(\mu_2), \quad \mu_1(?X) = \mu_2(?X)$$

- This means they assign the same value to any shared variables.

Example - Compatibility of Mappings

Consider the following SPARQL query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?student1 ?student2
WHERE {
    ?student1 ex:studyProgram ?program .
    ?student2 ex:studyProgram ?program .}
```

Example - Compatibility of Mappings

Consider the following SPARQL query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?student1 ?student2
WHERE {
    ?student1 ex:studyProgram ?program .
    ?student2 ex:studyProgram ?program .}
```

Mappings Example:

- $\mu_1(tp1) = \{(?student1, \mu_1(?student1)), (?program, \mu_1(?program))\}$
- $\mu_2(tp2) = \{(?student2, \mu_2(?student2)), (?program, \mu_2(?program))\}$
- $\text{dom}(\mu_1) = \{?student1, ?program\}$ and $\text{dom}(\mu_2) = \{?student2, ?program\}$
- μ_1 and μ_2 are compatible iff $\mu_1(?program) = \mu_2(?program)$

Example - JOIN Result for Compatible Mappings

Example of Mappings for Ω_1 and Ω_2 :

$\Omega_1 =$

$\{ \{ (?student1, ex:MarySmith), (?program, ex:ComputerScience) \},$
 $\{ (?student1, ex:JohnSmith), (?program, ex:ComputerScience) \} \}$

$\Omega_2 =$

$\{ \{ (?student2, ex:MarySmith), (?program, ex:ComputerScience) \},$
 $\{ (?student2, ex:JohnSmith), (?program, ex:ComputerScience) \} \}$

Example - JOIN Result for Compatible Mappings

Example of Mappings for Ω_1 and Ω_2 :

$\Omega_1 =$

$\{ \{ (?student1, ex:MarySmith), (?program, ex:ComputerScience) \},$
 $\{ (?student1, ex:JohnSmith), (?program, ex:ComputerScience) \} \}$

$\Omega_2 =$

$\{ \{ (?student2, ex:MarySmith), (?program, ex:ComputerScience) \},$
 $\{ (?student2, ex:JohnSmith), (?program, ex:ComputerScience) \} \}$

Result of Ω_1 JOIN Ω_2 :

Ω_1 JOIN $\Omega_2 =$

$\{ \{ (?student1, ex:MarySmith), (?student2, ex:MarySmith) \},$
 $\{ (?student1, ex:MarySmith), (?student2, ex:JohnSmith) \},$
 $\{ (?student1, ex:JohnSmith), (?student2, ex:MarySmith) \},$
 $\{ (?student1, ex:JohnSmith), (?student2, ex:JohnSmith) \} \}$

Understanding the Semantics of **FILTER**

Intuition

A **FILTER** restricts the mappings produced by a pattern to those that satisfy a given condition (e.g., a comparison or bound check).

Example:



A **FILTER** acts like a selection: it removes mappings that do not satisfy the condition.

Semantics of SPARQL - FILTER (1/3)

FILTER Semantics:

- The FILTER clause is used to apply constraints to the results of a pattern match.
- Semantics of a FILTERed pattern ($P_{\text{FILTER}} R$) is given by:

$$[[P_{\text{FILTER}} R]]_G = \{\mu \in [[P]]_G \mid \mu \models R\}$$

- Here, $\mu \models R$ denotes that the mapping μ satisfies the filter condition R .

Semantics of SPARQL - FILTER (2/3)

FILTER Conditions: The condition $\mu \models R$ holds if:

1. R is `bound(?X)` and $?X \in \text{dom}(\mu)$;
2. R is `?X = c`, $?X \in \text{dom}(\mu)$, and $\mu(?X) = c$;
3. R is `?X = ?Y`, $?X, ?Y \in \text{dom}(\mu)$, and $\mu(?X) = \mu(?Y)$;
4. R is $\neg R_1$ and it is not the case that $\mu \models R_1$;
5. R is $R_1 \vee R_2$ and $\mu \models R_1$ or $\mu \models R_2$;
6. R is $R_1 \wedge R_2$ and $\mu \models R_1$ and $\mu \models R_2$.

Semantics of SPARQL - FILTER (3/3)

Example:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?student1 ?student2
WHERE {
    ?student1 ex:studyProgram ?program .
    ?student2 ex:studyProgram ?program .
    FILTER (?student1 != ?student2)
}
```

Result:

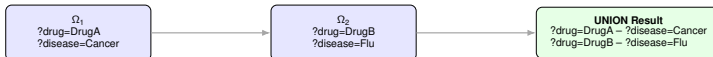
```
{{(?student1, ex:MarySmith), (?student2, ex:JohnSmith)},
 {(?student1, ex:JohnSmith), (?student2, ex:MarySmith)}}
```

Semantic Graph Patterns

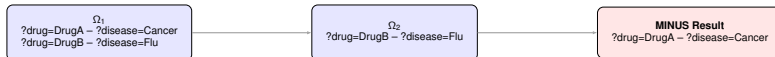
Intuition

Combination of mappings using **UNION**, **MINUS**, and **OPTIONAL**.

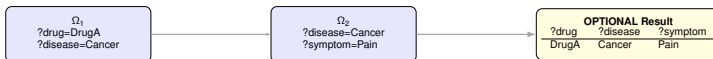
UNION: merge results



MINUS: remove overlaps



OPTIONAL: extend when possible



UNION merges, MINUS removes, OPTIONAL extends mappings.

Semantics of SPARQL Graph Patterns

Graph Pattern Semantics with UNION, MINUS, and OPTIONAL:

- ****UNION****: Combines results from two graph patterns.

$$\Omega_1 \text{ UNION } \Omega_2 = \{\mu \mid \mu \in \Omega_1 \text{ or } \mu \in \Omega_2\}$$

Semantics of SPARQL Graph Patterns

Graph Pattern Semantics with UNION, MINUS, and OPTIONAL:

- ****UNION****: Combines results from two graph patterns.

$$\Omega_1 \text{ UNION } \Omega_2 = \{\mu \mid \mu \in \Omega_1 \text{ or } \mu \in \Omega_2\}$$

- ****MINUS****: Excludes mappings from the first pattern that have overlapping variable bindings with the second.

$$\Omega_1 \text{ MINUS } \Omega_2 = \{\mu \mid \mu \in \Omega_1 \text{ and } \forall \mu' \in \Omega_2, \mu \text{ and } \mu' \text{ are incompatible}\}$$

Semantics of SPARQL Graph Patterns

Graph Pattern Semantics with UNION, MINUS, and OPTIONAL:

- ****UNION****: Combines results from two graph patterns.

$$\Omega_1 \text{ UNION } \Omega_2 = \{\mu \mid \mu \in \Omega_1 \text{ or } \mu \in \Omega_2\}$$

- ****MINUS****: Excludes mappings from the first pattern that have overlapping variable bindings with the second.

$$\Omega_1 \text{ MINUS } \Omega_2 = \{\mu \mid \mu \in \Omega_1 \text{ and } \forall \mu' \in \Omega_2, \mu \text{ and } \mu' \text{ are incompatible}\}$$

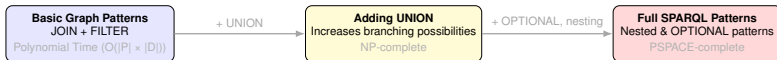
- ****OPTIONAL****: Includes mappings from the first pattern, optionally extending with those from the second pattern.

$$\Omega_1 \text{ OPTIONAL } \Omega_2 = (\Omega_1 \text{ JOIN } \Omega_2) \text{ UNION } (\Omega_1 \text{ MINUS } \Omega_2)$$

Understanding the Complexity of SPARQL Queries

Intuition

The more expressive the SPARQL query (i.e., the more operators it uses), the harder it is to compute its answers.



- **JOIN and FILTER:** easy to evaluate — similar to relational queries.
- **UNION:** adds nondeterministic choice, increasing search space.
- **OPTIONAL / nesting:** require reasoning about partial information — the hardest case.

Complexity grows with the expressive power of SPARQL's graph pattern language.

SPARQL Complexity

- **Theorem 1:** For a SPARQL query Q with only JOIN and FILTER operators in its Graph Pattern P :

Time Complexity: $O(|P| \times |D|)$

where D is the RDF dataset.

- **Theorem 2:** For a SPARQL query Q with Graph Pattern P using JOIN, FILTER, and UNION:

Complexity: NP-complete

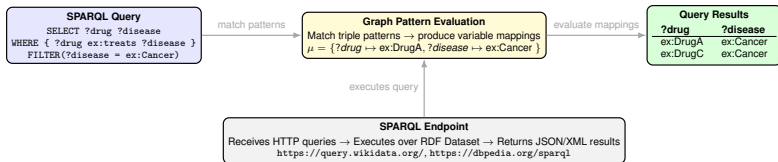
- **Theorem 3:** For a SPARQL query Q with unrestricted Graph Patterns:

Complexity: PSPACE-complete

- Complexity increases with query expressiveness, particularly when UNION and nested patterns are introduced.

Summary- SPARQL

SPARQL connects human-readable queries with machine-understandable RDF KGs.
It transforms graph patterns into mappings and delivers structured answers via SPARQL endpoints.



SPARQL bridges data and meaning — enabling access to RDF KGs.

Summary – SQL versus SPARQL

SQL	SPARQL
Based on relations (tables)	Based on edge-labeled graphs (RDF triples)
Queries combine tables using relational algebra operators	Queries combine triples using graph pattern matching
Tables to be joined must be specified explicitly	Operates over a default RDF graph , optionally extended via FROM clauses
Returns a new table (relation)	Returns a set of variable mappings (bindings)
Data described by schema and attributes	Data described by URIs, literals, and RDF properties

SQL queries structured data; SPARQL queries over knowledge graphs.

Typical Mistakes in SPARQL Queries

- Missing JOIN
- Misusing UNION
- Misusing OPTIONAL

Missing JOIN (1/2)

QS5- The courses taken by the Master students of LUH and the semester when the course was taken

Correct SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {?student      a                ex:MasterStudent.
       ?student      ex:university    ex:LUH.
       ?student      ex:hasTaken      ?courseTaken.
       ?courseTaken  ex:hasSemester   ?semester .
       ?courseTaken  ex:hasCourseId   ?course}
```

Missing JOIN (1/2)

QS5- The courses taken by the Master students of LUH and the semester when the course was taken

Correct SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {?student      a                ex:MasterStudent.
      ?student      ex:university    ex:LUH.
      ?student      ex:hasTaken      ?courseTaken.
      ?courseTaken  ex:hasSemester   ?semester .
      ?courseTaken  ex:hasCourseId   ?course}
```

Correct Answer

```
{{(?student,ex:JohnSmith),(?courseTaken,ex:hasTakenID1),
  (?semester,ex:SoSe2024),(?course,ex:DB101)}}
```

Missing JOIN (2/2)

QS5- The courses taken by the Master students of LUH and the semester when the course was taken

Incorrect SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {?student      a                ex:MasterStudent.
       ?student      ex:university    ex:LUH.
       ?courseTaken  ex:hasSemester   ?semester .
       ?courseTaken  ex:hasCourseId   ?course}
```

Missing JOIN (2/2)

QS5- The courses taken by the Master students of LUH and the semester when the course was taken

Incorrect SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {?student      a                      ex:MasterStudent.
       ?student      ex:university         ex:LUH.
       ?courseTaken  ex:hasSemester        ?semester .
       ?courseTaken  ex:hasCourseId        ?course}
```

Incorrect Answers

```
{({?student,ex:JohnSmith},{?courseTaken,ex:hasTakenID1},
{?semester,ex:SoSe2024},{?course,ex:DB101}),
{({?student,ex:JohnSmith},{?courseTaken,ex:hasTakenID2},
{?semester,ex:WiSe2024},{?course,ex:SDMKG}),
{({?student,ex:JohnSmith},{?courseTaken,ex:hasTakenID3},
{?semester,ex:SoSe2024},{?course,ex:SDMKG}),
{({?student,ex:MarySmith},{?courseTaken,ex:hasTakenID1},
{?semester,ex:SoSe2024},{?course,ex:DB101}),
{({?student,ex:MarySmith},{?courseTaken,ex:hasTakenID2},
{?semester,ex:WiSe2024},{?course,ex:SDMKG}),
{({?student,ex:MarySmith},{?courseTaken,ex:hasTakenID3},
{?semester,ex:SoSe2024},{?course,ex:SDMKG})}}
```

Error: Missing join between ?student and ?course

Missing UNION (1/2)

QS9 - For students of LUH or UniBonn, retrieve their study program, and if they are taking any course, retrieve the information of these courses

Correct SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{?student a ex:MasterStudent .
        ?student ex:university ex:LUH}
        UNION
        {?student a ex:MasterStudent .
        ?student ex:university ex:UniBonn}}
OPTIONAL
    {?student      ex:hasTaken      ?courseTaken .
     ?courseTaken ex:hasSemester ?semester .
     ?courseTaken ex:hasCourseId ?course .}}
```

Missing UNION (1/2)

QS9 - For students of LUH or UniBonn, retrieve their study program, and if they are taking any course, retrieve the information of these courses

Correct SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{{{?student a ex:MasterStudent .
        ?student ex:university ex:LUH}}
      UNION
        {{?student a ex:MasterStudent .
        ?student ex:university ex:UniBonn}}}
      OPTIONAL
        {{?student      ex:hasTaken      ?courseTaken .
        ?courseTaken ex:hasSemester ?semester .
        ?courseTaken ex:hasCourseId ?course .}}
```

Correct Answers

```
{{(?student,ex:JohnSmith),(?university, ex:LUH),(?courseTaken,ex:hasTakenID1),
(?semester,ex:SoSe2024),(?course,ex:DB101)},
{{(?student, ex:PeterHaas),(?university, ex:UniBonn),(?courseTaken, ex:hasTakenID2),
(?semester,ex:WiSe2024),(?course,ex:SDMKG)},
{{(?student, ex:PeterHaas),(?university, ex:UniBonn),(?courseTaken, ex:hasTakenID3),
(?semester,ex:SoSe2024),(?course,ex:SDMKG)},
{{(?student, ex:MarySmith), (?university, ex:LUH)}}
```

Missing UNION (2/2)

QS9 - For each Master student, retrieve the university where the student is registered, and if they are taking any course, the details of those courses.

Incorrect SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{?student a ex:MasterStudent .
        ?student ex:university ex:LUH
        ?student ex:university ex:UniBonn}
OPTIONAL
{{?student      ex:hasTaken      ?courseTaken .
  ?courseTaken ex:hasSemester ?semester .
  ?courseTaken ex:hasCourseId ?course .}}}
```

Missing UNION (2/2)

QS9 - For each Master student, retrieve the university where the student is registered, and if they are taking any course, the details of those courses.

Incorrect SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{?student a ex:MasterStudent .
        ?student ex:university ex:LUH
        ?student ex:university ex:UniBonn}
        OPTIONAL
        {?student      ex:hasTaken      ?courseTaken .
         ?courseTaken ex:hasSemester ?semester .
         ?courseTaken ex:hasCourseId ?course .}}}
```

Incorrect Answers

```
{}
```

Error: The JOIN requires that students belong to LUH and UniBonn.

Missing OPTIONAL (1/2)

QS8 - Retrieve the university where students are registered, and if they are taking any course, the information of these courses.

Correct SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{?student      a                ex:MasterStudent .
      ?student      ex:university ?university.}
      OPTIONAL
      {?student      ex:hasTaken      ?courseTaken.
      ?courseTaken ex:hasSemester ?semester.
      ?courseTaken ex:hasCourseId ?course.}}
```

Missing OPTIONAL (1/2)

QS8 - Retrieve the university where students are registered, and if they are taking any course, the information of these courses.

Correct SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {{?student      a                ex:MasterStudent .
      ?student      ex:university ?university.}
      OPTIONAL
      {{?student      ex:hasTaken      ?courseTaken.
        ?courseTaken ex:hasSemester ?semester.
        ?courseTaken ex:hasCourseId ?course.}}
```

Correct Answers

```
{{(?student,ex:JohnSmith),(?university, ex:LUH),(?courseTaken,ex:hasTakenID1),
(?semester,ex:SoSe2024),(?course,ex:DB101)},
{{(?student, ex:PeterHaas),(?university, ex:UniBonn),(?courseTaken, ex:hasTakenID2),
(?semester,ex:WiSe2024),(?course,ex:SDMKG)},
{{(?student, ex:PeterHaas),(?university, ex:UniBonn),(?courseTaken, ex:hasTakenID3),
(?semester,ex:SoSe2024),(?course,ex:SDMKG)},
{{(?student, ex:MarySmith), (?university, ex:LUH)}}
```

Note: OPTIONAL includes students (e.g., ex:MarySmith) not currently enrolled in any course.

Missing OPTIONAL (2/2)

QS9 - For each Master student, retrieve the university where the student is registered, and if they are taking any course, the details of those courses.

Incorrect SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {?student a ex:MasterStudent .
       ?student ex:university ?university.
       ?student ex:hasTaken ?courseTaken .
       ?courseTaken ex:hasSemester ?semester .
       ?courseTaken ex:hasCourseId ?course .}
```

Missing OPTIONAL (2/2)

QS9 - For each Master student, retrieve the university where the student is registered, and if they are taking any course, the details of those courses.

Incorrect SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT *
WHERE {?student a ex:MasterStudent .
       ?student ex:university ?university.
       ?student ex:hasTaken ?courseTaken .
       ?courseTaken ex:hasSemester ?semester .
       ?courseTaken ex:hasCourseId ?course .}
```

Incorrect Answers

```
{{(?student,ex:JohnSmith),(?university, ex:LUH),(?courseTaken,ex:hasTakenID1),
(?semester,ex:SoSe2024),(?course,ex:DB101)},
{(?student, ex:PeterHaas),(?university, ex:UniBonn),(?courseTaken, ex:hasTakenID2),
(?semester,ex:WiSe2024),(?course,ex:SDMKG)},
{(?student, ex:PeterHaas),(?university, ex:UniBonn),(?courseTaken, ex:hasTakenID3),
(?semester,ex:SoSe2024),(?course,ex:SDMKG)}}
```

Error: The JOIN requires all students to have course information, resulting in the exclusion of students not enrolled in courses.

References I



C. Gutiérrez, C. A. Hurtado, A. O. Mendelzon, and J. Pérez.
Foundations of semantic web databases.
J. Comput. Syst. Sci., 77(3):520–541, 2011.



A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. L. Gajo, R. Navigli, S. Neumaier, A. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. F. Sequeda, S. Staab, and A. Zimmermann.
Knowledge graphs.
ACM Comput. Surv., 54(4):71:1–71:37, 2022.



J. Pérez, M. Arenas, and C. Gutierrez.
Semantics and complexity of SPARQL.
ACM Trans. Database Syst., 34(3):16:1–16:45, 2009.